

DD2497 - Project Specification

Jonas D derman, Deniel Saskin, Mingtao Zhang, Ziyuan Jia
Group 1

November 2025

1 Introduction

This project aims to further expand S3K kernel by supporting filesystem encryption. The main purpose is to preserve the confidentiality of data if unauthorized physical access to the system has been obtained. Additionally, it helps by maintaining the integrity of persistent system states (e.g. user credentials) so that an attacker cannot modify it externally while keeping the system functional.

2 Goal

2.1 Minimum Goals

- FAT32 Filesystem Support for S3K
 - Implement file system functionality as isolated user-space services communicating over IPC.
 - Support reading, writing, creating, deleting and enumerating files and directories while complying to FAT32 metadata structure, allocation tables and file name rules.
 - Define basic crash-safety rules and write-ordering constraint appropriate for FAT32 using Rust.
- Filesystem-level Encryption
 - Implement the encryption functionality as an extension over the existing user-space filesystem service.
 - Supports AES-128-CBC encryption for file contents and AES-128-CTS-CBC encryption for filenames, as a standard approach.
 - Supports initializing the encryption on an unencrypted disk, and decryption for file access.
 - The final user manages the master key.

2.2 Optional Goals

- Additional Filesystem Support for S3K
 - Implement additional filesystem support (e.g. ext4) for S3K.
 - Support disk partition management and filesystem initialization.
- Filesystem-level Encryption
 - Evaluates the performance impact of using larger encryption keys, for example AES-256.
 - Supports key derivation and management for per-file keys.
 - Provides a secure method for the final user to recover the data if the master key is forgotten.

3 Workload

The project will involve the follow three major work areas.

3.1 Filesystem FAT32

Implementing block level I/O

3.2 Microkernel Integration

Designing IPC interface for file operations such as open, read, write, readdir, etc.

3.3 Encryption Layer

Using AES-128, the filesystem is able to encrypt files individually providing integrity of the files. The key in itself is a challenge to store as S3K doesn't provide safe storage of such valuable information. This problem will be discussed with supervisor.

4 Challenges

A majority of group members have previously never worked with micro kernels nor implementing new feature into an already existing kernel. Therefore, the research overhead required to conduct the project might be large but probably manageable. Furthermore, one group member has experience implementing FAT32 in his own micro kernel and might alleviate some pressure of the total workload of the project.